Building Archon: an Agentic Financial Intelligence Platform on Nebius Serverless AI

#NebiusServerlessChallenge · #ServerlessAI · #FinTech · #LLM

Small-business finance has a quiet failure mode. The numbers look clean while the underlying documents disagree. A company's financial truth is scattered across dozens of document types: sales and purchase invoices, orders, receipts, payments, bank transfers and statements, payroll, expenses. Each one is entered, mis-entered, or never entered by a different hand.

Archon pulls all of it into one place. It produces a consolidated, period-over-period view (P&L, EBITDA, per-period metrics, total workforce cost, cash), then cross-checks the whole picture to surface what is missing or does not reconcile. It reads scanned and digital documents in several languages, and writes an executive summary on top.

This post is about the engineering. Three parts in particular: the document-fusion insight the product is built around, the multi-agent architecture on **Nebius Serverless AI**, and the trust design that lets a language model near a financial report without ever touching the arithmetic.

> **Architecture diagram:** the full component graph (frontend, BFF, CPU endpoint, jobs, storage, DB, inference) is rendered in the repository README.

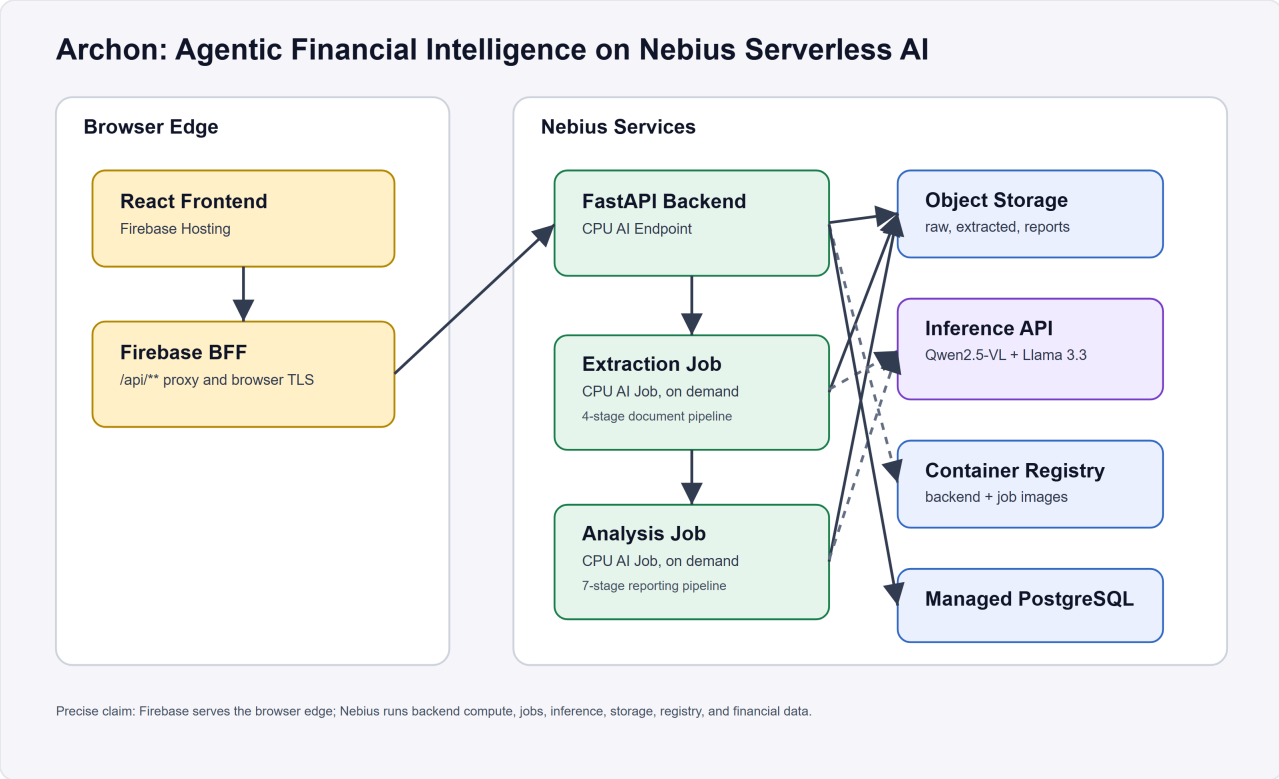

Figure 1. Archon spans a Firebase public edge and Nebius backend services.

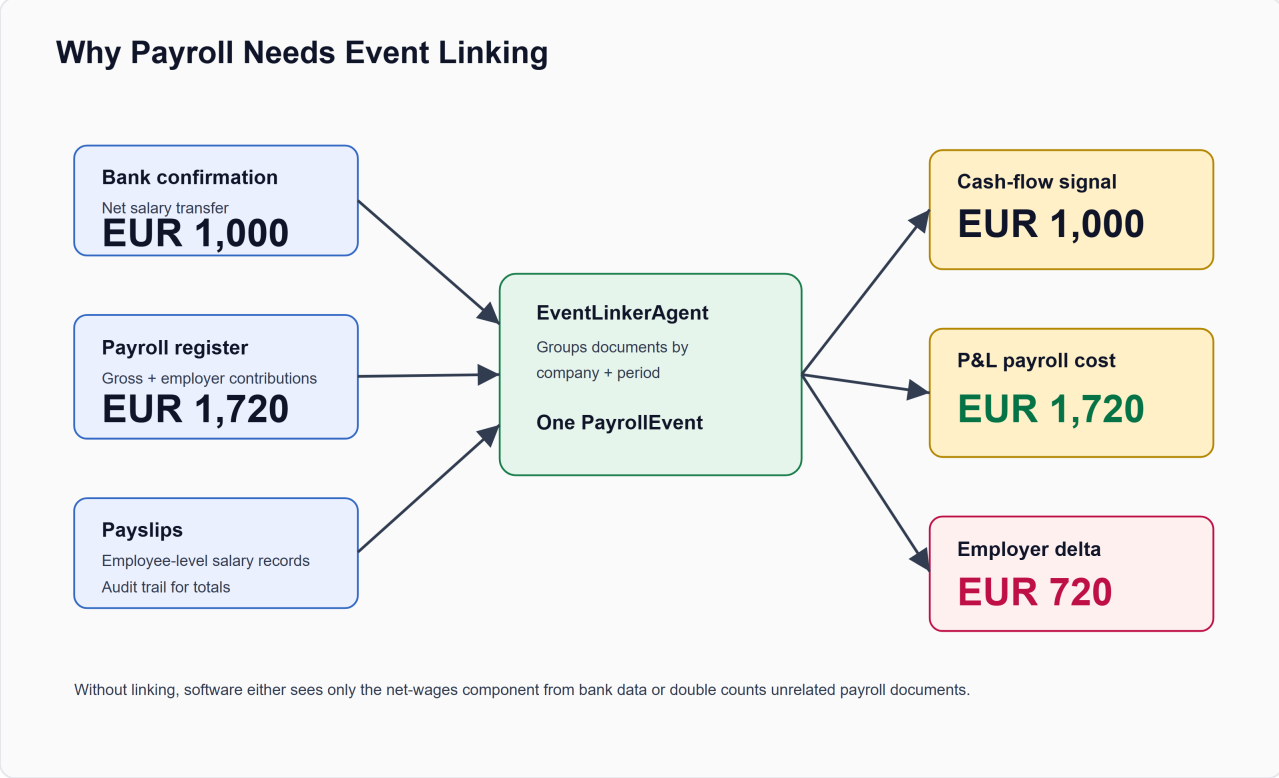

Figure 2. Three payroll documents fused into one event, reconciling the full employer cost (about 72% above the bank transfer).

The insight: one event, three documents, three different truths

The clearest example of "the documents disagree" is a single payroll event. It produces three artifacts, and each one tells a different part of the truth:

Document	What it reports	Illustrative figures (one employee)
Bank confirmation	Net salary transferred to the employee	net wages
Payslip	Gross − employee contributions − tax	net wages, gross pay
Payroll register	Gross + **employer** contributions	full employer cost

The register's true cost of employment is the net wages, plus the tax and social security withheld from the employee, plus the employer's own contributions. The bank debit shows only the net-wages component. So software that reads only the bank statement sees only that component. On the sample, the register's true employer cost reconciles to about **72% above** what left the bank. The employer's own social-security contribution alone is about 35% of the transfer, and the rest is the withheld employee tax and social security. For most SMBs, payroll is the single largest cost centre.

No single document can be trusted alone. The fix is to *fuse* the three into one event and read the right figure for the right question. That job belongs to a dedicated agent:

```
# jobs/extraction/agents/event_linker.py
def _build_event(company, period, docs):
    bank      = _pick_one(docs, DocType.BANK_CONFIRMATION)
    register  = _pick_one(docs, DocType.PAYROLL_REGISTER)
    payslips  = [d for d in docs if d.doc_type == DocType.PAYSLIP]

    is_complete = all([bank is not None, register is not None, len(payslips) > 0])
    return PayrollEvent(period=period, company_name=company or None,
                        bank_confirmation=bank, payroll_register=register,
                        payslips=payslips, is_complete=is_complete)
```

Once the event is linked, the P&L uses the register's employer cost and the cash-flow view uses the bank transfer. It is the same event, counted once, correctly, from two angles. The same reconciliation shape extends to vendors. A `ReconciliationAgent` flags invoices a vendor statement references but the system never received.

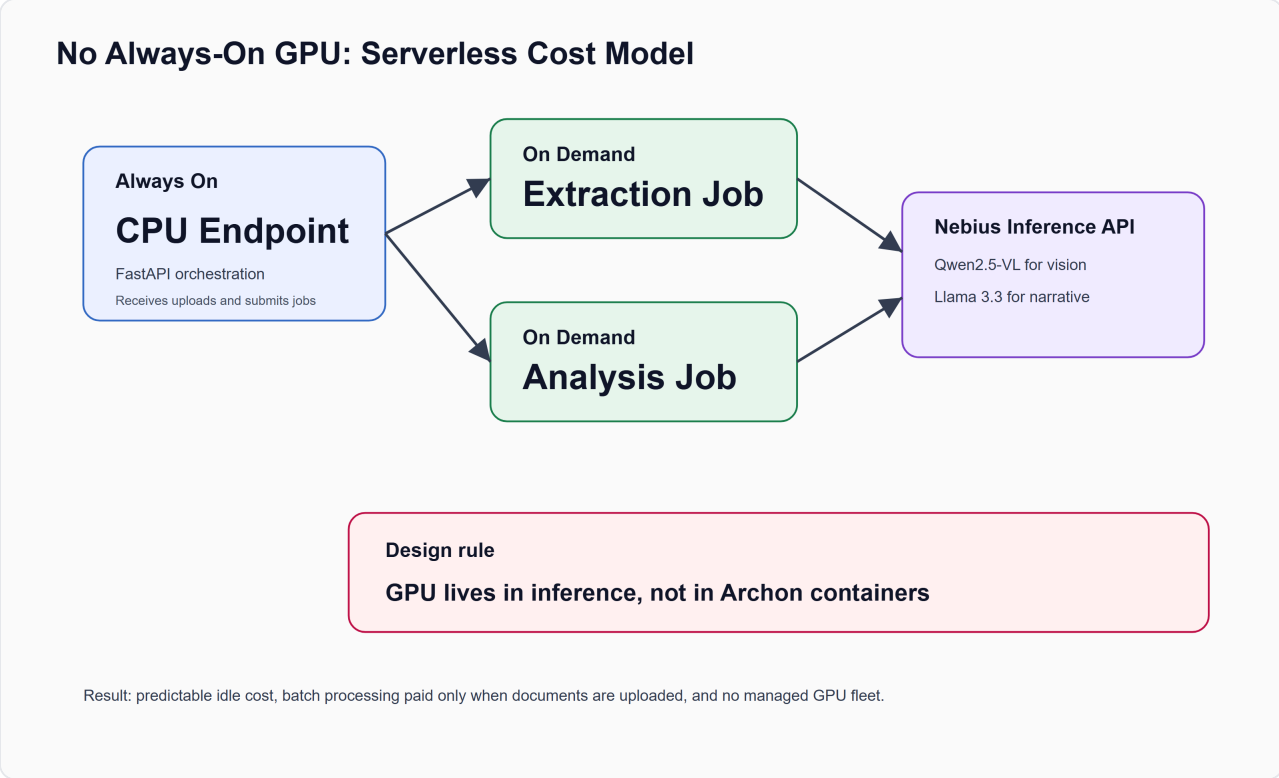

Figure 4. CPU endpoint and CPU jobs call the Nebius Inference API instead of an always-on GPU.

Why Nebius Serverless AI fit the workload

The workload is bursty. A customer uploads documents once a month, waits for processing, then may not run another batch for weeks. That is a poor fit for an always-on GPU. Archon uses three Nebius compute primitives instead:

- a **CPU AI Endpoint** for the always-on FastAPI orchestration backend (`/upload · /jobs · /analyze · /reports`);
- a **CPU AI Job for extraction** that starts on upload, processes the batch, writes JSON, and self-terminates;
- a **CPU AI Job for analysis** that reads the extracted JSON, builds the report, and self-terminates.

The decisive choice is that **the GPU is not inside Archon's containers**. Extraction and analysis are cheap CPU Python containers that call the **Nebius Inference API** over HTTP: Qwen2.5-VL-72B for vision extraction, Llama-3.3-70B for narration. The frontier models live in Nebius's inference layer, so Archon's own containers stay disposable. The only always-on cost is a ~$0.04/hr CPU endpoint, and each job run costs about a cent. Object Storage holds the raw, extracted, and report artifacts (`documents.json` , `events.json` , `validation.json`). Managed PostgreSQL holds the indexed `documents` records, queryable per period, doc-type, and upload. Container Registry hosts the three images. Six Nebius services, one workflow.

The React frontend and a thin BFF route sit on Firebase for public hosting, login, and browser-edge TLS. The honest claim is precise: **all domain compute and stateful financial infrastructure run on Nebius, and Firebase is only the public edge.**

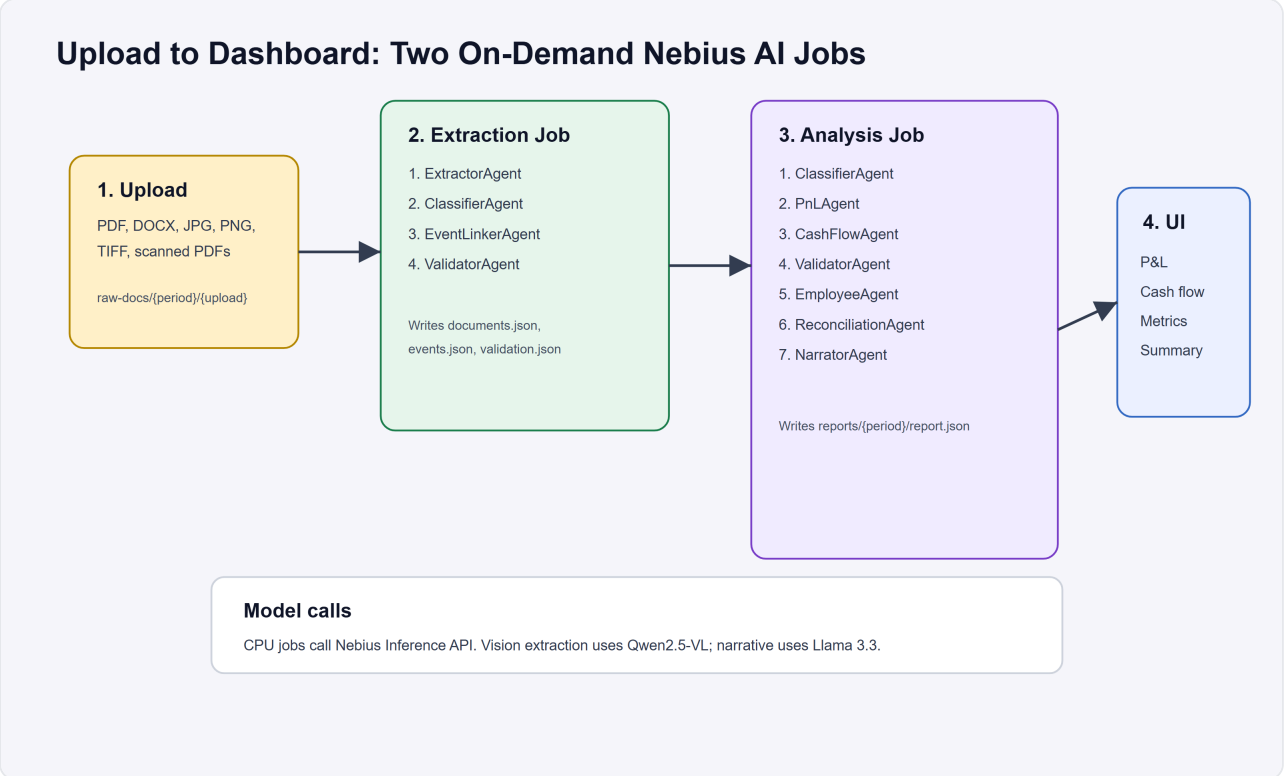

Figure 3. Upload, extraction, analysis, dashboard: the on-demand job pipeline.

Two agent pipelines

Extraction (4 agents) turns raw files into structured JSON. `ExtractorAgent` auto-detects file type, routing digital text to text extraction and scans or images to the vision model. `ClassifierAgent` then *deterministically* refines the document type, which keeps common LLM misclassifications out of the accounting layer. `EventLinkerAgent` fuses the payroll triad described above. `ValidatorAgent` runs cross-document consistency checks.

Analysis (7 agents) turns that JSON into a dashboard-ready report. It re-classifies, then runs `PnLAgent`, `CashFlowAgent`, a `ValidatorAgent` cross-document safety net, `EmployeeAgent` (whose payroll-event summaries consume that validation), `ReconciliationAgent`, and finally `NarratorAgent` for the executive summary. Each agent has a single responsibility, which makes it easy to test and easy to reason about.

Trust: the numbers are deterministic, the LLM only narrates

For a financial product, "a language model computed your P&L" is a non-starter. Archon is built so it never does. Every figure (P&L, expense breakdown, vendor summaries, key metrics) is pure Python arithmetic:

```
# jobs/analysis/agents/pnl_agent.py
"""
PnLAgent — aggregates extracted documents into P&L metrics.
Single responsibility: pure Python arithmetic over classified documents.
No LLM call; deterministic and fast.
"""
def build_pnl(period, docs):
    revenue  = sum(d.total_amount for d in docs if d.doc_type in REVENUE_DOC_TYPES)
    expenses = _compute_expenses(docs)
    net_profit = revenue - expenses
    return MonthlyPnL(period=period, revenue=round(revenue, 2),
                      expenses=round(expenses, 2), netProfit=round(net_profit, 2), ...)
```

The only place a model touches the analysis is `NarratorAgent`, and it writes a three-to-four-sentence summary *from the already-computed metrics*. If that call fails, the report still renders. The narrative is the garnish, not the meal. The numbers you see are not hallucinated. They are `round(sum(...), 2)`.

The cross-document checks are equally auditable. `ValidatorAgent` runs four named, deterministic rules with explicit tolerances: `R1: bank.total ≈ Σ payslips ±2%`, `R2: employer_cost / net_pay ∈ [1.40, 2.60]`, `R3: bank date ≤ period end`, `R4: register headcount == payslip count`. Every flag cites the rule, the two figures compared, and the source files. It is a finding you can check by hand, not "the model thought something looked off."

Measuring it, and an honest caveat

A claim like "the true payroll cost reconciles to ~72% over the bank net" is worth nothing without a number behind it. So the repo ships an **evaluation harness** (`eval/`) that scores the *real* pipeline agents against a labelled synthetic corpus. It runs offline, with no API key, using only `pydantic`:

```
python eval/generate_corpus.py && python eval/evaluate.py
```

On the deterministic 40-case corpus, under perfect extraction, the `PnLAgent` reports employer cost to the cent, at **100% field and fusion accuracy**. The register's true employer cost reconciles to the register total, about **72% over the naive bank-only view on the sample**. Every component is tied to a source document. The thesis is verified, not asserted.

The uncomfortable first result was the whole reason to build a harness. Two of the four validation rules were **dormant**. R2 and R4 fired 0/37 times because they read fields (`employer_cost_total`, `net_pay_total`, `employee_count`) the extraction prompt never requested. The harness turned that from an unknown into a measured 0/37, with file-and-line evidence and a one-prompt fix. We wired those fields into the extractor, and the same harness now measures **37/37**. R2 and R4 fire on every applicable case, proven before and after rather than asserted. The full write-up is in `eval/BASELINE.md`. Finding it before a customer does is the whole point.

One lesson worth keeping: a serverless job can lie to you

A Nebius AI Job is a *request* for compute, not a guarantee. Quota is granted per compute preset, and when a preset has zero quota the platform does something worse than reject you. It *accepts* the job, strands it in `PROVISIONING`, never allocates an instance, and tears it down with no error. Your submission returned a job id. Nothing is coming.

Archon now treats this as a first-class failure mode. It submits, probes for a real instance within a bounded window, and on a *never-provisioned* outcome it deletes the stalled job and climbs a config-driven preset ladder. It fails over only when a job never got compute, never when a job reached compute and then crashed (that second bug would recur on every rung). If every rung fails, the API returns one actionable `503` instead of a silent spinner. The full taxonomy is in `docs/capacity-probe-pattern.md`, reproducible offline with `bash scripts/demo-failover.sh`.

There is a second failure mode this exposed, and it is worth being upfront about. AI Jobs are the primary design. The submission path, the pysdk integration, and the capacity probe are all built and tested. But this tenant's `cpu-d3` AI-Jobs quota is a hard **0**, verified empirically across every preset and region, so no Job ever provisions. Rather than gate the live demo on a quota grant, Archon carries a runtime fallback. Set `JOB_RUNNER_BACKEND=inline` and the *same* extraction and analysis pipelines run as isolated subprocesses inside the Endpoint, tracked in Object Storage exactly like a Job. It uses subprocess isolation rather than in-process import, because the two job packages have colliding top-level module names. The pipeline completes end to end either way. The honest summary: the Jobs integration is real and ships in the image, and the inline runner is the resilience path that keeps the product working when the platform grants zero Jobs capacity.

One more detail bit us. When the backend submits a job, Nebius must pull the image, and registry credentials belong on the job spec itself as a **single** message, not a list:

```
# backend/services/nebius.py
registry_credentials=JobSpec.RegistryCredentials(username="iam", password=token)
```

Treating it as repeated turns job submission into a 500. Small detail, real outage.

Making a best-effort path loud

Archon mirrors every finished report into Managed PostgreSQL as a relational read-model. The mirror is best-effort by design. Object Storage is the source of truth, so if the database write ever fails the report still renders. That design has a quiet downside. Best-effort code fails silently, and a mirror that does nothing looks exactly like a mirror that works.

So database reachability became a first-class signal. The backend exposes a small `/health/db` probe that opens a connection and runs `SELECT 1`. The deploy pipeline calls it the moment the endpoint goes live and writes the verdict into the job summary. If PostgreSQL is unreachable, the deploy says so in plain language, instead of leaving a dead mirror for someone to notice weeks later. The connection runs over the cluster's private in-VPC endpoint, so it never depends on a public allowlist that shifts every time the endpoint is recreated.

Proving the write used to need a full analysis run, and a full run needs job quota. To break that dependency the repo ships a one-command seed workflow. It writes a valid report to storage for a throwaway period, calls the read path, and watches the relational tables populate. Anyone can verify the database path end to end without spending a cent of compute.

The pattern under this and the capacity probe is the same. A serverless system has more ways to fail quietly than a single machine does. The engineering worth keeping is whatever turns each quiet failure into a loud one.

Try it

The local stack needs no Nebius account, just an Inference API key. LocalStack stands in for object storage, and jobs run as local containers:

```
git clone https://github.com/upgradedev/archon_nebius && cd archon_nebius
cp .env.example .env          # set NEBIUS_INFERENCE_API_KEY
docker compose up --build
bash scripts/test-pipeline.sh # drives the full pipeline, prints the report JSON
```

Or see the payoff with zero setup. **https://archon-pnl.web.app/?demo=1** renders a full sample report (P&L, charts, validations, executive summary) entirely client-side, with no backend call.

That is the point of the build. Once Nebius handles the serverless compute and inference surfaces, the hard work moves back where it belongs, to domain correctness.

Built for the Nebius Serverless AI Builders Challenge 2026. Code: https://github.com/upgradedev/archon_nebius (MIT).